

Project Report #8

Documentation Agent

AI Generated Draft Gemini service is currently under heavy load. Project Context: You are a Senior Technical Writer. Project: akash Architecture: # AI Generated Draft Gemini service is currently under heavy load. Project Context: You are a Senior Software Architect. Project: akash Generate: 1. System Architecture 2. Database Design 3. API Design 4. Folder Structure 5. Recommended Architecture: * Modular Backend * REST APIs * Secure Authentication * Agent Based Workflow * Scalable Deployment Fallback response generated successfully.

Research Agent

AI Generated Draft Gemini service is currently under heavy load. Project Context: You are a Senior Software Architect. Analyze the following project: akash Generate: 1. Functional Requirements 2. Non Functional Requirements 3. Recommended Tech Stack 4. Challenges 5. Development Roadmap Recommended Architecture: * Modular Backend * REST APIs * Secure Authentication * Agent Based Workflow * Scalable Deployment Fallback response generated successfully.

Architect Agent

AI Generated Draft Gemini service is currently under heavy load. Project Context: You are a Senior Software Architect. Project: akash Generate: 1. System Architecture 2. Database Design 3. API Design 4. Folder Structure 5. Deployment Architecture Recommended Architecture: * Modular Backend * REST APIs * Secure Authentication * Agent Based Workflow * Scalable Deployment Fallback response generated successfully.

Coding Agent

As a Senior Software Architect, I've analyzed the "Akash" project, a decentralized cloud computing marketplace. My recommendations are guided by the principles of a modular backend, REST APIs, secure authentication, agent-based workflow, and scalable deployment. --- ## 1. Functional Requirements The Akash project aims to create a marketplace where users (Tenants) can deploy applications on a decentralized network of compute providers. 1. **Provider Management:** * **Provider Registration:** Allow individuals or organizations to register as compute providers on the network, specifying their available resources (CPU, RAM, storage, GPU, IP addresses, bandwidth). * **Resource Listing & Updates:** Enable providers to list their current resource availability and update it in real-time. * **Provider Profile Management:** Allow providers to manage their public profile, pricing strategies, and service agreements. * **Provider Monitoring:** Monitor the health and performance of provider nodes. 2. **Tenant (Deployer) Management:** * **User Registration & Authentication:** Securely onboard new users and authenticate existing ones. * **Deployment Creation:** Allow tenants to define their application's resource requirements and deployment manifest (e.g., using Akash SDL). * **Deployment Lifecycle Management:** Enable tenants to start, stop, update, scale, and terminate their deployed applications. * **Deployment Monitoring & Logging:** Provide tenants with real-time access to logs and performance metrics of their applications. 3. **Marketplace & Leasing Engine:** * **Resource Discovery:** Efficiently match tenant deployment requests with available provider resources. * **Bidding System:** Facilitate a bidding process where providers offer prices for tenant deployments. * **Lease Management:** Create, manage, and close leases between tenants and providers upon bid acceptance. * **Price**

Discovery:** Mechanisms for transparent pricing and competitive bidding. 4. **Billing & Payments:**

- * **Akash Token (AKT) Integration:** Use AKT for all transactions (bids, payments, settlements).
- * **Wallet Management:** Provide functionalities for users to manage their AKT wallet, including deposits, withdrawals, and balance inquiries.
- * **Payment Escrow & Settlement:** Securely handle payments, ensuring providers are paid for resources consumed and tenants are only charged for what they use.

5. **Network & Security:**

- * **Decentralized Network Interaction:** Core interaction with the Akash blockchain for all critical operations (deployments, leases, payments).
- * **Secure Communication:** Ensure encrypted and authenticated communication between tenants, providers, and the marketplace.
- * **Identity Management:** Robust identity verification for both providers and tenants (potentially integrating with decentralized identity solutions).

6. **User Interface (UI) / User Experience (UX):**

- * **Intuitive Dashboard:** A web-based dashboard for both tenants and providers to manage their activities.
- * **CLI Tool:** A command-line interface for advanced users and automation.
- * **Notifications:** Alert users about important events (e.g., new bids, deployment status changes).

--- ## 2. Non-Functional Requirements

1. **Scalability:**

- * **High Throughput:** Support a large volume of deployment requests, bids, and blockchain transactions.
- * **Elasticity:** The backend services should be able to scale horizontally to accommodate fluctuating load.
- * **Provider & Tenant Capacity:** Support thousands of providers and hundreds of thousands of active deployments.

2. **Reliability & Availability:**

- * **Fault Tolerance:** The system should remain operational even if individual components (e.g., a single provider, a backend service instance) fail.
- * **Disaster Recovery:** Robust backup and recovery mechanisms for critical data.
- * **High Uptime:** Target 99.9% uptime for the core marketplace services.

3. **Performance:**

- * **Low Latency:** Fast response times for API requests, especially for bidding and deployment creation.
- * **Efficient Resource Matching:** Quick matching of tenant requests with suitable providers.
- * **Real-time Monitoring:** Near real-time data collection and display for monitoring and logging.

4. **Security:**

- * **Data Protection:** End-to-end encryption for data in transit and at rest.
- * **Access Control:** Role-based access control (RBAC) to ensure users only access authorized resources.
- * **Vulnerability Management:** Regular security audits, penetration testing, and prompt patching of vulnerabilities.
- * **Blockchain Security:** Secure interaction with the Akash blockchain, protecting private keys and signing operations.

5. **Decentralization:**

- * **Censorship Resistance:** The core compute layer should be resistant to censorship or single points of control.
- * **Transparency:** All on-chain transactions and agreements should be verifiable.

6. **Usability:**

- * **Developer Experience:** Clear API documentation, SDKs, and intuitive CLI for developers.
- * **User-Friendly UI:** Simple and intuitive web interface for non-technical users.

7. **Maintainability & Extensibility:**

- * **Modular Architecture:** Easily add new features or modify existing ones without impacting the entire system.
- * **Clean Codebase:** Well-documented code, unit, and integration tests.
- * **Observability:** Comprehensive monitoring, logging, and tracing to quickly identify and diagnose issues.

--- ## 3. Recommended Tech Stack Given the project's nature and the hint towards FastAPI:

Backend & Core Services:

- * **Primary Languages:** Python (for API services, data processing), Go (for blockchain interaction, high-performance agents).
- * **Web Framework:** **FastAPI (Python):** For building high-performance REST APIs due to its modern features, async support, and Pydantic integration.
- * **Database:** **PostgreSQL:** Robust, scalable relational database for structured marketplace data (users, providers, deployments, leases metadata).
- * **Redis:** For caching, session management, and rate limiting.
- * **Message Queue/Event Bus:** **Kafka / RabbitMQ:** For asynchronous processing, event-driven architecture, and decoupling services (e.g., for processing blockchain events, long-running deployment tasks).
- * **Blockchain Interaction:** **Go SDK for Cosmos/Tendermint:** For direct interaction with the Akash blockchain.
- * **Python Libraries:** **`grpcio`** for gRPC communication with Tendermint/Cosmos SDK.
- * **Containerization:** **Docker:** For packaging applications and their dependencies.
- * **Orchestration:** **Kubernetes:** For deploying, scaling, and managing containerized applications in a highly available manner.

Frontend (Web Application):

- * **Framework:** **React / Next.js (TypeScript):** For building a dynamic, interactive, and performant user interface. Next.js offers server-side rendering (SSR) for better SEO and performance.
- * **State Management:** **Zustand / React Query:** Lightweight and efficient state management.
- * **Styling:** **Tailwind CSS / Material-UI:** For rapid UI development and consistent design.
- * **API Client:** **Axios / Fetch API:** For making HTTP requests to the backend.
- * **Provider Agent (runs on provider machines):** **Go:** (for efficiency, low resource consumption, concurrency).
- * **Tools:** Docker, Kubernetes (if provider hosts a cluster), or direct system-level access to manage compute

resources. **DevOps & Infrastructure:** **Cloud Platform:** AWS / GCP / Azure (for hosting the centralized backend components and managed services). **CI/CD:** **GitHub Actions / GitLab CI:** For automated testing, building, and deployment pipelines. **Monitoring & Logging:** **Prometheus & Grafana** (for metrics), **ELK Stack** (Elasticsearch, Logstash, Kibana) / **Grafana Loki** (for logs): For comprehensive observability. **API Gateway/Reverse Proxy:** **Nginx / Traefik:** For external routing, load balancing, and SSL termination. --- ## 4. Challenges

- Blockchain Integration Complexity:** **Asynchronous Operations:** Interacting with a blockchain often involves waiting for transaction confirmations, requiring careful design of asynchronous workflows and event handling. **State Synchronization:** Keeping the centralized database state synchronized with the decentralized blockchain state can be challenging. **Error Handling:** Robust error handling for blockchain-related failures (e.g., network congestion, gas limit issues).
- Key Management:** Securely managing cryptographic keys for signing blockchain transactions.
- Decentralized Resource Orchestration:** **Heterogeneous Environments:** Providers can have diverse hardware, network configurations, and operating systems, making standardized deployment and monitoring difficult. **Security at the Edge:** Ensuring the security of agents running on untrusted provider machines. **Network Latency & Reliability:** Managing deployments across a geographically dispersed and potentially unreliable network of providers.
- Scalability & Performance:** **Bidding System Load:** A highly active marketplace could generate a large number of bids, requiring an efficient and scalable bidding engine. **Monitoring Data Ingestion:** Collecting and processing metrics/logs from thousands of deployments across many providers.
- User Experience:** **Abstracting Blockchain Complexity:** Making the decentralized nature of Akash accessible and easy to use for mainstream developers who may not be familiar with blockchain concepts. **Debugging Decentralized Deployments:** Providing adequate tools for tenants to debug issues with their applications running on remote, decentralized providers.
- Security & Trust:** **Smart Contract Vulnerabilities:** The core logic is governed by smart contracts; bugs here can have catastrophic consequences. **Malicious Providers:** Guarding against providers who might attempt to tamper with deployments, steal data, or provide subpar service. **Sybil Attacks:** Preventing single entities from controlling multiple provider identities to manipulate the marketplace.
- Dispute Resolution:** **Automated vs. Manual:** Designing fair and efficient mechanisms to resolve disputes between tenants and providers (e.g., provider uptime, resource allocation).

--- ## 5. Development Roadmap This roadmap outlines key phases, with iterations within each phase.

Phase 1: Core Marketplace MVP (3-6 Months)

- Foundation & Infrastructure Setup:** Set up core CI/CD pipelines, Dockerfiles, Kubernetes manifests. Establish cloud infrastructure for backend services (DB, Message Queue, K8s cluster).
- Basic API Gateway setup.**
- Backend Core Services:**
 - User & Auth Service:** Implement user registration, login (JWT), basic profile management.
 - Provider Service:** Basic provider registration (name, region, Akash address), ability to announce generic resource capacity.
 - Deployment Service:** Tenants can submit a basic SDL manifest, store it.
 - Blockchain Integration (Minimal):** Ability to generate Akash addresses, submit basic deployment requests to testnet.
 - Database Schema (Initial):** `users`, `providers`, `deployments`, `leases`.
 - CLI & Basic Frontend:** Basic CLI for providers to register and tenants to deploy. Simple React frontend for user login, displaying user's deployments.
 - Agent Prototype:** A simple Go agent that can run on a provider machine, capable of receiving a deployment command and reporting basic status.

Phase 2: Enhanced Marketplace & Automation (6-12 Months)

- Refined Backend Services:**
 - Resource Specification:** Allow providers to detail resource configurations (CPU, RAM, storage, specific software versions).
 - Bidding & Leasing Engine:** Implement the core bidding logic, matching, and lease creation via blockchain.
 - Payment Service:** Integrate AKT wallet functionalities (deposit/withdraw), automatic lease payments.
 - Monitoring & Logging Integration:** Backend endpoints to aggregate logs/metrics from provider agents.
 - Blockchain Integration (Advanced):** Handle lease state changes, disputes, payments fully on-chain.
 - Asynchronous Processing:** Leverage Kafka/RabbitMQ for processing blockchain events and long-running tasks.
 - Web Dashboard (MVP):**
 - User-friendly dashboard for tenants to create/manage deployments, view bids, accept leases.
 - Provider dashboard for listing resources, managing bids, viewing active leases.
 - Wallet UI for AKT balance and transaction history.
 - Agent Refinement:** Enhanced Go agent for managing Docker/Kubernetes deployments on provider nodes, collecting metrics, streaming logs. Secure communication between agent and central services.
 - Scalability & Performance Tuning:** Load testing and optimization of API services

and database. * Initial auto-scaling configurations for Kubernetes deployments. **Phase 3: Ecosystem & Advanced Features (12-18+ Months)** * **Advanced Networking:** * Support for custom domains, private networking between deployments, advanced firewall rules. * **Dispute Resolution System:** * Implement mechanisms for submitting and resolving disputes (e.g., automated checks, community-driven resolution). * **Developer Tooling & SDKs:** * Publish comprehensive API documentation, SDKs in various languages, and examples. * Integrations with popular CI/CD tools. * **Enhanced Security:** * Implement multi-factor authentication, advanced threat detection. * Regular security audits and bounty programs. * **Monitoring & Observability:** * Full-fledged Prometheus/Grafana and ELK stack deployment. * Distributed tracing. * **Community & Governance Features:** * Allow community proposals, voting for protocol upgrades. * Analytics and reporting for marketplace activity. * **Edge Cases & Resiliency:** * Refine error handling, graceful degradation, and system recovery procedures. --- ## Backend Folder Structure ``` akash-backend/
 ├── app/
 │ ├── main.py # FastAPI application entry point
 │ ├── api/ # API endpoint definitions (routers)
 │ ├── __init__.py
 │ └── v1/ # API Version 1
 ├── __init__.py
 ├── auth.py # Authentication & user registration
 ├── users.py # User profile management
 ├── providers.py # Provider listing and management
 ├── deployments.py # Tenant deployment creation and management
 ├── leases.py # Lease management (accepting bids, closing)
 ├── wallet.py # AKT wallet operations (deposit, withdraw, balance)
 ├── core/ # Core application configurations and utilities
 ├── __init__.py
 ├── config.py # Pydantic BaseSettings for environment variables
 ├── security.py # JWT token handling, password hashing, OAuth2
 ├── dependencies.py # Dependency injection for database sessions, current user
 ├── db/ # Database interaction layer
 ├── __init__.py
 ├── database.py # SQLAlchemy engine and session management
 ├── models.py # SQLAlchemy ORM models (table definitions)
 ├── migrations/ # Alembic migration scripts
 ├── crud.py # CRUD operations for database models
 ├── schemas/ # Pydantic models for request/response validation
 ├── __init__.py
 ├── user.py # User schemas (UserCreate, UserResponse, etc.)
 ├── provider.py
 ├── deployment.py
 ├── lease.py
 ├── wallet.py
 ├── auth.py # Auth schemas (Token, LoginRequest, etc.)
 ├── services/ # Business logic and external integrations
 ├── __init__.py
 ├── blockchain_service.py # Logic for interacting with the Akash blockchain
 ├── deployment_orchestrator.py # Manages lifecycle of deployments on provider agents
 ├── monitoring_service.py # Integrates with external monitoring systems
 ├── payment_service.py # Handles payment processing logic
 ├── notification_service.py # Sending emails, push notifications
 ├── tests/ # Unit and integration tests
 ├── __init__.py
 ├── test_auth.py
 ├── test_deployments.py
 ├── confest.py # Pytest fixtures
 ├── utils/ # General utility functions (e.g., date formats, helpers)
 ├── __init__.py
 ├── helpers.py
 ├── Dockerfile # Docker build instructions for the application
 ├── requirements.txt # Python dependencies
 ├── alembic.ini # Alembic configuration file
 ├── pyproject.toml # Poetry/Flit configuration (or setup.py)
 ├── README.md ``` --- ## Frontend Folder Structure ``` akash-frontend/
 ├── public/
 ├── index.html # Main HTML file
 ├── favicon.ico
 ├── src/
 │ ├── App.tsx # Main application component
 │ ├── index.tsx # Entry point for React app
 │ ├── assets/ # Static assets (images, icons, fonts)
 │ ├── images/
 │ ├── svg/
 │ ├── components/ # Reusable UI components
 │ ├── common/ # Generic components (Button, Modal, Input, Spinner)
 │ ├── layout/ # Layout components (Header, Footer, Sidebar, Navbar)
 │ ├── domain/ # Domain-specific components (ProviderCard, DeploymentStatusBadge, BidList)
 │ ├── hooks/ # Custom React hooks for reusable logic
 │ ├── useAuth.ts
 │ ├── useDeployments.ts
 │ ├── useMediaQuery.ts
 │ ├── pages/ # Top-level page components (views corresponding to routes)
 │ ├── Auth/ # Authentication related pages
 │ ├── LoginPage.tsx
 │ ├── RegisterPage.tsx
 │ ├── Dashboard/ # User-specific dashboards
 │ ├── TenantDashboard.tsx
 │ ├── ProviderDashboard.tsx
 │ ├── Deployments/ # Deployment management pages
 │ ├── DeploymentListPage.tsx
 │ ├── DeploymentDetailPage.tsx
 │ ├── Providers/ # Provider listing and details
 │ ├── ProviderListPage.tsx
 │ ├── ProviderDetailPage.tsx
 │ ├── Wallet/ # Wallet and transaction pages
 │ ├── WalletPage.tsx
 │ ├── NotFoundPage.tsx # 404 page
 │ ├── services/ # API interaction layer
 │ ├── authService.ts
 │ ├── providerService.ts
 │ ├── deploymentService.ts
 │ ├── walletService.ts
 │ ├── api.ts # Axios/Fetch instance configuration
 │ ├── store/ # State management (e.g., Zustand/Redux slices)
 │ ├── index.ts
 │ ├── authStore.ts # Auth state
 │ ├── deploymentStore.ts #

Deployment-related state ■ ■ ■ ■ ■ uiStore.ts # UI-specific state (e.g., theme, notifications) ■ ■ ■ ■ ■
 styles/ # Global styles, theme, utility classes ■ ■ ■ ■ ■ index.css ■ ■ ■ ■ ■ variables.css # CSS
 variables ■ ■ ■ ■ ■ theme.ts # Theme configuration (for Material-UI, Chakra UI etc.) ■ ■ ■ ■ ■ types/
 # TypeScript interfaces and types for data structures ■ ■ ■ ■ ■ api.ts # API request/response types
 ■ ■ ■ ■ ■ auth.ts ■ ■ ■ ■ ■ common.ts ■ ■ ■ ■ ■ models.ts # Core data models (User, Provider,
 Deployment etc.) ■ ■ ■ ■ ■ utils/ # General utility functions ■ ■ ■ ■ ■ validators.ts ■ ■ ■ ■ ■
 helpers.ts ■ ■ ■ ■ ■ routes.tsx # React Router configuration ■ ■ ■ ■ ■ .env # Environment variables ■ ■ ■ ■ ■
 package.json ■ ■ ■ ■ ■ tsconfig.json # TypeScript configuration ■ ■ ■ ■ ■ yarn.lock (or package-lock.json)
 ■ ■ ■ ■ ■ README.md ``` --- ## Database Schema (PostgreSQL) ```sql -- Extension for UUIDs
 CREATE EXTENSION IF NOT EXISTS "uuid-oss"; -- Users Table CREATE TABLE users (id
 UUID PRIMARY KEY DEFAULT uuid_generate_v4(), username VARCHAR(50) UNIQUE NOT
 NULL, email VARCHAR(100) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL,
 role VARCHAR(20) NOT NULL DEFAULT 'tenant' CHECK (role IN ('tenant', 'provider', 'admin')),
 akash_address VARCHAR(255) UNIQUE, -- User's associated Akash blockchain address
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP, updated_at
 TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP); -- Providers Table
 CREATE TABLE providers (id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), user_id UUID
 UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE, name VARCHAR(100)
 NOT NULL, description TEXT, region VARCHAR(50), status VARCHAR(20) NOT NULL DEFAULT
 'active' CHECK (status IN ('active', 'inactive', 'suspended')), created_at TIMESTAMP WITH TIME
 ZONE DEFAULT CURRENT_TIMESTAMP, updated_at TIMESTAMP WITH TIME ZONE
 DEFAULT CURRENT_TIMESTAMP); -- Provider Resources Table (Capabilities offered by a
 provider, potentially dynamic) -- Note: Actual resource pricing and availability is typically managed
 on-chain by Akash. -- This table stores a cached/snapshot view for marketplace search. CREATE
 TABLE provider_resources (id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), provider_id
 UUID NOT NULL REFERENCES providers(id) ON DELETE CASCADE, cpu_total_units INTEGER,
 memory_total_gb NUMERIC(10, 2), storage_total_gb NUMERIC(10, 2), gpu_total_count
 INTEGER, ip_addresses_total INTEGER, attributes JSONB, -- e.g., {"region": "us-east-1", "tier":
 "premium"} last_updated TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP); --
 Deployments Table (Tenant's application deployments) CREATE TABLE deployments (id UUID
 PRIMARY KEY DEFAULT uuid_generate_v4(), tenant_id UUID NOT NULL REFERENCES
 users(id) ON DELETE CASCADE, name VARCHAR(100) NOT NULL, manifest_url TEXT NOT
 NULL, -- URL to the Akash SDL manifest akash_dseq BIGINT UNIQUE, -- Decentralized Sequence
 Number from Akash blockchain status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK
 (status IN ('pending', 'active', 'suspended', 'closed', 'failed')), created_at TIMESTAMP WITH TIME
 ZONE DEFAULT CURRENT_TIMESTAMP, updated_at TIMESTAMP WITH TIME ZONE
 DEFAULT CURRENT_TIMESTAMP); -- Leases Table (Agreement between a tenant and a
 provider) CREATE TABLE leases (id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
 deployment_id UUID NOT NULL REFERENCES deployments(id) ON DELETE CASCADE,
 provider_id UUID NOT NULL REFERENCES providers(id) ON DELETE CASCADE,
 akash_lease_id TEXT UNIQUE, -- Unique ID for the lease on Akash blockchain (e.g.,
 dseq/gseq/oseq) price_per_unit NUMERIC(20, 10) NOT NULL, -- Price in AKT per time unit
 price_denom VARCHAR(10) NOT NULL DEFAULT 'uakt', -- Denomination (e.g., uakt) status
 VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'active', 'closed',
 'settled', 'failed')), start_time TIMESTAMP WITH TIME ZONE, end_time TIMESTAMP WITH TIME
 ZONE, created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP, UNIQUE
 (deployment_id, akash_lease_id) -- A deployment can have multiple lease records if lease
 changes. akash_lease_id is unique per actual lease.); -- Bids Table (Optional: Cache recent bids,
 but core bidding is on-chain) CREATE TABLE bids (id UUID PRIMARY KEY DEFAULT
 uuid_generate_v4(), deployment_id UUID NOT NULL REFERENCES deployments(id) ON
 DELETE CASCADE, provider_id UUID NOT NULL REFERENCES providers(id) ON DELETE
 CASCADE, akash_bid_id TEXT UNIQUE, -- Unique ID for the bid on Akash blockchain
 price_per_unit NUMERIC(20, 10) NOT NULL, price_denom VARCHAR(10) NOT NULL DEFAULT
 'uakt', status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending',
 'accepted', 'rejected', 'withdrawn')), created_at TIMESTAMP WITH TIME ZONE DEFAULT
 CURRENT_TIMESTAMP); -- Transactions Table (For tracking user deposits, withdrawals,

```

payments) CREATE TABLE transactions ( id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, type VARCHAR(20)
NOT NULL CHECK (type IN ('deposit', 'withdrawal', 'lease_payment', 'escrow_funding')), amount
NUMERIC(25, 10) NOT NULL, currency VARCHAR(10) NOT NULL DEFAULT 'AKT', tx_hash
VARCHAR(255) UNIQUE, -- Blockchain transaction hash status VARCHAR(20) NOT NULL
DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed', 'failed')), created_at TIMESTAMP
WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP ); -- Indexes for performance CREATE
INDEX idx_users_email ON users(email); CREATE INDEX idx_users_akash_address ON
users(akash_address); CREATE INDEX idx_providers_user_id ON providers(user_id); CREATE
INDEX idx_deployments_tenant_id ON deployments(tenant_id); CREATE INDEX
idx_deployments_akash_dseq ON deployments(akash_dseq); CREATE INDEX
idx_leases_deployment_id ON leases(deployment_id); CREATE INDEX idx_leases_provider_id
ON leases(provider_id); CREATE INDEX idx_leases_akash_lease_id ON leases(akash_lease_id);
CREATE INDEX idx_bids_deployment_id ON bids(deployment_id); CREATE INDEX
idx_transactions_user_id ON transactions(user_id); CREATE INDEX idx_transactions_tx_hash ON
transactions(tx_hash); `` -- ## API Endpoints (RESTful, JSON) **Base URL:** /api/v1` #### 1.
Authentication & User Management * **POST /auth/register** * **Description:** Register a new
user (tenant or provider). * **Request:** `{ "username": "...", "email": "...", "password": "...", "role":
"tenant/provider" }` * **Response:** `{ "access_token": "...", "token_type": "bearer", "user": { ... } }` *
**POST /auth/login** * **Description:** Authenticate user and get access token. * **Request:** `{
"username": "...", "password": "..."}` * **Response:** `{ "access_token": "...", "token_type": "bearer",
"user": { ... } }` * **POST /auth/refresh-token** * **Description:** Get a new access token using a
refresh token (if implemented). * **GET /users/me** * **Description:** Get current user's profile. *
**Auth:** Required (JWT) * **PUT /users/me** * **Description:** Update current user's profile. *
**Auth:** Required (JWT) #### 2. Provider Management * **POST /providers** * **Description:**
Register a user as a provider (linked to `users.id`). * **Auth:** Required (JWT, user must have
'provider' role) * **Request:** `{ "name": "My Data Center", "description": "...", "region": "us-west-1"
}` * **Response:** `{ "id": "...", "name": "...", ... }` * **GET /providers** * **Description:** List all
registered providers (can include filtering/pagination). * **GET /providers/{provider_id}` *
**Description:** Get details of a specific provider. * **PUT /providers/{provider_id}` *
**Description:** Update provider profile (only by owner or admin). * **Auth:** Required (JWT) *
**POST /providers/{provider_id}/resources** * **Description:** Announce or update available
resources for a provider (triggers on-chain transaction). * **Auth:** Required (JWT, owner) *
**Request:** `{ "cpu_cores": 16, "memory_gb": 64, "storage_gb": 1024, "gpu_count": 0,
"ip_addresses": 1, "attributes": { "region": "us-east-1" } }` #### 3. Deployment Management
(Tenants) * **POST /deployments** * **Description:** Create a new deployment request by
providing an Akash SDL manifest. This initiates the on-chain deployment creation. * **Auth:**
Required (JWT, 'tenant' role) * **Request:** `{ "name": "my-web-app", "manifest_url":
"https://example.com/my-app.sdl" }` * **Response:** `{ "id": "...", "name": "...", "status": "pending",
"akash_dseq": 12345, ... }` * **GET /deployments** * **Description:** List all deployments
belonging to the current tenant. * **Auth:** Required (JWT) * **GET
/deployments/{deployment_id}` * **Description:** Get details of a specific deployment. * **Auth:**
Required (JWT, owner) * **PUT /deployments/{deployment_id}` * **Description:** Update a
deployment (e.g., manifest URL, scaling). Triggers new on-chain transaction. * **Auth:** Required
(JWT, owner) * **DELETE /deployments/{deployment_id}` * **Description:** Terminate a
deployment. Triggers on-chain transaction to close the deployment. * **Auth:** Required (JWT,
owner) * **GET /deployments/{deployment_id}/logs** * **Description:** Retrieve real-time or
historical logs for a deployment. * **Auth:** Required (JWT, owner) * **GET
/deployments/{deployment_id}/metrics** * **Description:** Retrieve monitoring metrics for a
deployment (CPU, RAM, network). * **Auth:** Required (JWT, owner) #### 4. Bids & Leases *
**GET /deployments/{deployment_id}/bids** * **Description:** For a tenant, list bids received for
their deployment request. * **Auth:** Required (JWT, tenant owner) * **POST
/deployments/{deployment_id}/bids/{bid_id}/accept** * **Description:** Tenant accepts a specific
bid, initiating a lease on-chain. * **Auth:** Required (JWT, tenant owner) * **GET /leases** *
**Description:** List all active/historical leases for the current user (either as tenant or provider). *
**Auth:** Required (JWT) * **GET /leases/{lease_id}` * **Description:** Get details of a specific
lease. * **Auth:** Required (JWT, involved party) * **PUT /leases/{lease_id}/close** *

```

****Description:**** Close an active lease. ****Auth:**** Required (JWT, involved party) **### 5. Wallet & Payments**

****GET /wallet/balance**** ****Description:**** Get the current AKT balance for the authenticated user's associated Akash address. ****Auth:**** Required (JWT)

****POST /wallet/deposit**** ****Description:**** Initiate a deposit process (e.g., provide a deposit address). ****Auth:**** Required (JWT)

****POST /wallet/withdraw**** ****Description:**** Request a withdrawal of AKT. ****Auth:**** Required (JWT) ****Request:**** `{ "amount": 100, "recipient_address": "akash1..." }`

****GET /wallet/transactions**** ****Description:**** List transaction history for the user's wallet. ****Auth:**** Required (JWT)

--- **### Sample FastAPI Code** Here's a sample for `main.py` and `api/v1/deployments.py` illustrating the structure and interaction with a hypothetical `blockchain_service`.

```

**1. `akash-backend/app/main.py`
python # akash-backend/app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from .api.v1 import auth, users, providers, deployments, leases, wallet
from .core.config import settings
# from .db.database import init_db_and_create_tables
# For local dev/testing if not using Alembic
app = FastAPI(
    title=settings.PROJECT_NAME,
    version=settings.API_VERSION,
    description="Akash Decentralized Cloud Marketplace API"
)
# CORS middleware for frontend development
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    # Adjust this to your frontend URL in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
# Include routers
app.include_router(auth.router, prefix="/api/v1/auth", tags=["Auth"])
app.include_router(users.router, prefix="/api/v1/users", tags=["Users"])
app.include_router(providers.router, prefix="/api/v1/providers", tags=["Providers"])
app.include_router(deployments.router, prefix="/api/v1/deployments", tags=["Deployments"])
app.include_router(leases.router, prefix="/api/v1/leases", tags=["Leases"])
app.include_router(wallet.router, prefix="/api/v1/wallet", tags=["Wallet"])

@app.get("/")
async def root():
    return {"message": "Welcome to Akash Backend API"}

@app.on_event("startup")
async def startup_event():
    print("Application starting up...")
    # await init_db_and_create_tables()
    # Uncomment for initial table creation without Alembic
    # Initialize blockchain connection or other services
    # Example: await blockchain_service.connect()

@app.on_event("shutdown")
async def shutdown_event():
    print("Application shutting down...")
    # Clean up resources
    # Example: await blockchain_service.disconnect()

**2. `akash-backend/app/api/v1/deployments.py`
(Note: This sample assumes `crud.py`, `models.py`, `schemas/deployment.py`, `schemas/user.py`, `blockchain_service.py` and `deployment_orchestrator.py` are already defined according to the folder structure.)
python # akash-backend/app/api/v1/deployments.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List
from ...db.database import get_db
from ...db import crud
from ...schemas import deployment as deployment_schemas, user as user_schemas
from ...core.security import get_current_user
from ...services import blockchain_service, deployment_orchestrator

router = APIRouter()

@router.post("/", response_model=deployment_schemas.DeploymentResponse, status_code=status.HTTP_201_CREATED)
async def create_deployment(
    deployment_create: deployment_schemas.DeploymentCreate,
    current_user: user_schemas.User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    """Allows a tenant to create a new deployment request on the Akash network. This involves storing the request locally and initiating an on-chain deployment.
    if current_user.role != "tenant":
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Only tenants can create deployments")
    if not current_user.akash_address:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="User does not have an associated Akash address.")

    # 1. Store deployment request metadata in our centralized DB
    db_deployment = crud.create_deployment(
        db=db,
        tenant_id=current_user.id,
        name=deployment_create.name,
        manifest_url=deployment_create.manifest_url,
        status="pending"
    )
    # 2. Interact with Akash blockchain service to create the deployment on-chain
    # This call would typically involve signing a transaction with the tenant's key.
    try:
        # Assuming blockchain_service handles the actual protobuf conversion and transaction signing/broadcast
        akash_dseq = await blockchain_service.create_akash_deployment(
            tenant_address=current_user.akash_address,
            sdl_manifest_url=deployment_create.manifest_url,
            deployment_name=deployment_create.name
            # Optional, for logging/tracking on blockchain service
        )
        # Update our DB with the on-chain deployment sequence ID and status
        db_deployment = crud.update_deployment_dseq_and_status(
            db=db,
            deployment_id=db_deployment.id,
            akash_dseq=akash_dseq,
            status="active"
        )
    except Exception as e:
        # If blockchain interaction fails, mark the deployment as

```

```

failed and raise an error crud.update_deployment_status(db=db,
deployment_id=db_deployment.id, status="failed") raise HTTPException(
status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=f"Failed to create
deployment on Akash network: {e}" ) return db_deployment @router.get("/{",
response_model=List[deployment_schemas.DeploymentResponse]) async def read_deployments(
skip: int = 0, limit: int = 100, current_user: user_schemas.User = Depends(get_current_user), db:
Session = Depends(get_db) ): """ Retrieves a list of deployments for the current authenticated user.
""" deployments = crud.get_user_deployments(db, user_id=current_user.id, skip=skip, limit=limit)
return deployments @router.get("/{deployment_id}",
response_model=deployment_schemas.DeploymentResponse) async def
read_deployment_by_id( deployment_id: str, # Assuming UUID string current_user:
user_schemas.User = Depends(get_current_user), db: Session = Depends(get_db) ): """ Retrieves
details of a specific deployment by its ID. """ db_deployment = crud.get_deployment_by_id(db,
deployment_id=deployment_id) if not db_deployment: raise
HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Deployment not found") if
db_deployment.tenant_id != current_user.id: raise
HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Not authorized to access
this deployment") return db_deployment @router.delete("/{deployment_id}",
status_code=status.HTTP_204_NO_CONTENT) async def terminate_deployment( deployment_id:
str, current_user: user_schemas.User = Depends(get_current_user), db: Session =
Depends(get_db) ): """ Terminates an active deployment on the Akash network. """ db_deployment
= crud.get_deployment_by_id(db, deployment_id=deployment_id) if not db_deployment: raise
HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Deployment not found") if
db_deployment.tenant_id != current_user.id: raise
HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Not authorized to terminate
this deployment") if db_deployment.status == "closed": raise
HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Deployment is already
closed.") # Interact with Akash blockchain to close the deployment try: if
db_deployment.akash_dseq: await blockchain_service.close_akash_deployment(
tenant_address=current_user.akash_address, akash_dseq=db_deployment.akash_dseq ) #
Update local DB status after successful blockchain interaction
crud.update_deployment_status(db=db, deployment_id=deployment_id, status="closed") else: # If
dseq is not set, maybe it failed before reaching blockchain, just mark locally
crud.update_deployment_status(db=db, deployment_id=deployment_id, status="failed") except
Exception as e: raise HTTPException(
status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=f"Failed to terminate
deployment on Akash network: {e}" ) return # Placeholder for monitoring and logging endpoints
@router.get("/{deployment_id}/logs") async def get_deployment_logs( deployment_id: str,
current_user: user_schemas.User = Depends(get_current_user), db: Session = Depends(get_db) ):
""" Retrieves logs for a given deployment. This would typically query a log aggregation service. """
db_deployment = crud.get_deployment_by_id(db, deployment_id=deployment_id) if not
db_deployment or db_deployment.tenant_id != current_user.id: raise
HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Not authorized or
deployment not found.") # Logic to fetch logs from a monitoring_service or direct logging endpoint
(e.g., Loki/Elasticsearch) logs = await deployment_orchestrator.get_deployment_logs(
deployment_id=deployment_id, akash_dseq=db_deployment.akash_dseq ) return
{"deployment_id": deployment_id, "logs": logs, "message": "Logs retrieved successfully."}
@router.get("/{deployment_id}/metrics") async def get_deployment_metrics( deployment_id: str,
current_user: user_schemas.User = Depends(get_current_user), db: Session = Depends(get_db) ):
""" Retrieves metrics for a given deployment. This would typically query a metric aggregation
service (e.g., Prometheus). """ db_deployment = crud.get_deployment_by_id(db,
deployment_id=deployment_id) if not db_deployment or db_deployment.tenant_id !=
current_user.id: raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Not
authorized or deployment not found.") # Logic to fetch metrics from a monitoring_service or direct
metrics endpoint (e.g., Prometheus) metrics = await
deployment_orchestrator.get_deployment_metrics( deployment_id=deployment_id,
akash_dseq=db_deployment.akash_dseq ) return {"deployment_id": deployment_id, "metrics":

```



```
metrics, "message": "Metrics retrieved successfully."} ````
```

Testing Agent

As a Senior QA Engineer, I will design a comprehensive testing strategy for the Akash project, ensuring all functional and non-functional requirements are thoroughly validated. My approach will cover multiple layers of the application stack, from individual units to end-to-end user flows, and will integrate security and performance considerations throughout the development lifecycle. --- ## Testing Plan for Akash Project

****Overall Testing Goals:****

- * Verify that all functional requirements for Tenant, Provider, Marketplace, and Billing are met.
- * Ensure the system meets defined non-functional requirements (Scalability, Reliability, Performance, Security, Decentralization, Usability, Maintainability).
- * Identify and mitigate risks associated with blockchain integration, decentralized orchestration, and security.
- * Establish an efficient and reliable automated testing pipeline.

--- ### 1. Unit Test Cases

****Objective:**** Verify the correctness of individual functions, methods, and classes in isolation.

****Scope:****

- * Backend Python (FastAPI): Business logic in ``services/``, CRUD operations in ``db/crud.py``, Pydantic schemas, utility functions, security helpers.
- * Go Agent: Resource discovery, reporting, container management logic, secure communication.
- * Frontend (React/Next.js): Individual React components, custom hooks, utility functions.

****Examples (Backend - Python FastAPI):****

- * ``test_verify_password_correct()``: Verify correct password matches hashed password.
- * ``test_verify_password_incorrect()``: Verify incorrect password does not match.
- * ``test_create_access_token()``: Ensure JWT token is generated with correct payload and expiration.
- * ``test_get_current_user_valid_token()``: Authenticate a valid token and retrieve user.
- * ``test_get_current_user_invalid_token()``: Handle expired/invalid tokens gracefully.
- * ``test_create_user_success()``: Create a new user record.
- * ``test_get_user_by_email()``: Retrieve user by email.
- * ``test_create_deployment_success()``: Create a new deployment record.
- * ``test_update_deployment_status()``: Update the status of an existing deployment.
- * ``test_get_user_deployments()``: Fetch deployments filtered by user ID.

****Edge Case:****

- * ``test_create_user_duplicate_email()``: Ensure database constraint violation is handled (e.g., raises error or returns None).

****Frontend (React Components - using Jest/React Testing Library):****

- * ``test_login_form_renders_correctly()``: Check if all input fields and buttons are present.
- * ``test_deployment_card_displays_data()``: Verify a ``DeploymentCard`` component renders with given deployment data.
- * ``test_button_onClick_event()``: Ensure a button's click handler is invoked.

****Tools:****

- * **Python:** ``pytest``, ``unittest``, ``pytest-mock`` (for mocking dependencies like database or external services).
- * **Go:** Built-in ``go test`` framework.
- * **Frontend:** ``Jest``, ``React Testing Library``.

--- ### 2. Integration Test Cases

****Objective:**** Verify that different modules or services interact correctly with each other and with external systems.

****Scope:****

- * Backend API (FastAPI) <-> PostgreSQL Database
- * Backend API <-> Akash Blockchain (Testnet/Devnet, or well-defined mock)
- * Backend API <-> Message Queue (Kafka/RabbitMQ)
- * Backend API <-> Provider Agent communication
- * Frontend <-> Backend API
- * Monitoring/Logging Service <-> Backend/Provider Agent

****Examples:****

- * **User Registration & Login (API <-> DB):**
 - * ``test_register_user_and_login()``: Register a new user via API, then attempt to log in with credentials and verify JWT token.
 - * ``test_register_provider_flow()``: Register a user with 'provider' role, then create a provider profile linked to that user.
- * **Deployment Lifecycle (API <-> DB <-> Blockchain Service):**
 - * ``test_create_deployment_end_to_end()``: Tenant calls ``POST /deployments``, verify deployment record is created in DB and a (mocked/testnet) blockchain transaction is initiated successfully, updating ``akash_dseq`` in DB.

`test\_terminate\_deployment\_flow()`: Tenant calls `DELETE /deployments/{id}`, verify DB status changes and (mocked/testnet) blockchain transaction to close deployment. * **Error Handling:** *
`test\_create\_deployment\_blockchain\_failure\_rolls\_back\_status()`: If blockchain transaction fails, ensure DB deployment status is updated to 'failed'. * **Bidding & Leasing (API <-> DB <-> Blockchain Service):** *
`test\_provider\_receives\_deployment\_request\_and\_bids()`: Simulate a deployment request, then a provider's bid (via API or direct blockchain interaction), verify bid appears for the tenant. *
`test\_tenant\_accepts\_bid\_creates\_lease()`: Tenant accepts a bid via API, verify a lease record is created in DB and a (mocked/testnet) blockchain lease is established. *
Provider Agent Communication (Agent <-> Backend API): *
`test\_agent\_registers\_resources\_via\_api()`: Provider agent sends resource data to `POST /providers/{id}/resources`, verify DB update. *
`test\_agent\_streams\_logs\_to\_backend()`: Provider agent streams log data, backend service ingests it into monitoring/logging system, and `GET /deployments/{id}/logs` retrieves it. *
`test\_agent\_reports\_metrics\_to\_backend()`: Provider agent reports metrics, backend ingests, `GET /deployments/{id}/metrics` retrieves it. *
Frontend-Backend Integration: *
`test\_dashboard\_loads\_user\_deployments()`: Log in via UI, navigate to dashboard, ensure user's deployments are fetched and displayed correctly from the backend API. *
`test\_deployment\_form\_submission()`: Fill out deployment form on UI, submit, verify successful API call and UI update. *
Tools: * **Python:** * `pytest`, `requests` (for API calls), `SQLAlchemy` (for direct DB assertions in tests), `Docker Compose` (to spin up dependent services like DB, message queue, mocked blockchain). * **Go:** * `go test`, HTTP clients. *
Containerization: * Docker Compose for managing test environments. * **Blockchain Simulators/Testnets:** * Local Akash node, or public testnets. --- **### 3. API Test Cases**
Objective: * Validate the functionality, reliability, performance, and security of the REST API endpoints. This focuses on the external contract of the APIs. **Scope:** * All defined API endpoints. **Examples (based on the provided API Endpoints section):** * **Functional Validations:** *
POST /auth/register *
* Valid registration (tenant, provider role). *
* Registration with existing email/username (expect 409 Conflict or 400 Bad Request). *
* Registration with invalid input (e.g., weak password, malformed email) (expect 422 Unprocessable Entity). * **POST /auth/login** *
* Successful login with correct credentials. *
* Login with incorrect password/username (expect 401 Unauthorized). * **GET /users/me** *
* Access with valid token (expect 200 OK, user details). *
* Access without token or with invalid token (expect 401 Unauthorized). * **POST /providers** *
* User with 'provider' role registers as a provider successfully. *
* User with 'tenant' role attempts to register as provider (expect 403 Forbidden). * **POST /deployments** *
* Create deployment with valid SDL URL and tenant credentials (expect 201 Created, `akash\_dseq` present). *
* Create deployment with invalid SDL URL (expect 400 Bad Request or 422 Unprocessable Entity). *
* Create deployment without associated Akash address (expect 400 Bad Request). * **DELETE /deployments/{deployment_id}** *
* Tenant terminates their own deployment successfully. *
* Tenant attempts to terminate another tenant's deployment (expect 403 Forbidden). *
* Terminate already closed deployment (expect 400 Bad Request). * **POST /deployments/{deployment_id}/bids/{bid_id}/accept** *
* Tenant accepts a valid bid for their deployment. *
* Tenant accepts a bid for another tenant's deployment (expect 403 Forbidden). *
* Accept a non-existent or already accepted bid. * **Edge Cases & Error Handling:** * **Invalid IDs:** *
* Accessing resources with malformed or non-existent UUIDs/IDs (e.g., `GET /providers/invalid-uuid`). * **Empty Payloads:** *
* Sending empty or partially empty request bodies when required. * **Rate Limiting:** *
* Test API behavior when hitting configured rate limits (expect 429 Too Many Requests). * **Data Consistency:** *
* After an action (e.g., deployment creation), verify the data is consistently reflected in subsequent `GET` requests. * **Pagination & Filtering:** *
* Test `GET /providers` and `GET /deployments` with various `skip`, `limit`, and potential filter parameters. *
Authorization (RBAC): *
* Ensure a 'tenant' cannot access 'provider'-only endpoints or data, and vice-versa (unless explicitly allowed, e.g., viewing public provider profiles). *
* Ensure users can only modify/delete their own resources. * **Tools:** * **Postman** / **Newman** (for manual and automated API collections). *
* `pytest` with `httpx` or `requests` library (for programmatic API testing within the Python test suite). *
* `curl` (for quick manual tests). --- **### 4. Security Test Cases**
Objective: *
* Identify vulnerabilities, ensure data protection, and verify access control mechanisms. **Scope:** *
* All backend services, frontend, blockchain interactions, and infrastructure. **Examples:** *
Authentication & Authorization: * **Brute-Force Protection:** *
* Implement and test rate limiting on login attempts. * **Session Management:** *
* Test JWT token expiration, invalidation, and secure

storage (on frontend). * **Role-Based Access Control (RBAC):** Exhaustively test that each user role can only perform authorized actions (e.g., a tenant cannot register a provider, a provider cannot view another provider's private data). * **Token Hijacking:** Test if stolen tokens can be used after logout or expiration. * **Input Validation & Sanitization (OWASP Top 10):** * **Injection Flaws (SQL Injection, Command Injection):** Test all input fields (usernames, descriptions, manifest URLs, etc.) for malicious input. * **Cross-Site Scripting (XSS):** Test UI for rendering user-supplied input without proper sanitization. * **Broken Access Control:** Verify users cannot directly access resources they are not authorized for by manipulating URLs or request parameters (e.g., `GET /deployments/{other\_user\_id\_deployment}`). * **Insecure Deserialization:** Ensure data transmitted (especially from provider agents) is safely deserialized. * **Data Protection:** * **Encryption In Transit:** Verify all API communication uses HTTPS/TLS 1.2+ (e.g., using `SSL Labs` scanner). * **Encryption At Rest:** Verify sensitive data in PostgreSQL (e.g., password hashes) is properly stored and that database backups are encrypted. * **Sensitive Data Exposure:** Ensure no sensitive information (private keys, full passwords, internal errors) is exposed in API responses, logs, or UI. * **Blockchain Interaction Security:** * **Key Management:** How are private keys stored and used for signing transactions? Test that they are never exposed, and only used for authorized actions. * **Transaction Integrity:** Verify that transactions broadcast to Akash are correctly signed and match the intended payload. * **Replay Attacks:** Ensure mechanisms are in place to prevent replay of blockchain transactions. * **Decentralized Security Concerns:** * **Malicious Provider Simulation:** * Provider reports fake resource availability/metrics. How does the system detect/respond? * Provider fails to deploy an accepted lease. How is this detected? * Provider attempts to access tenant data. * **Sybil Attacks:** Test mechanisms to prevent a single entity from registering multiple provider identities to manipulate the marketplace. * **DDoS Resistance:** How well does the centralized backend withstand attacks on its services? * **Dependency Security:** * **Vulnerability Scanning:** Regularly scan all third-party libraries and dependencies for known vulnerabilities (e.g., using `Snyk`, `Dependabot`). * **Container Image Scanning:** Scan Docker images for vulnerabilities before deployment. * **Logging & Monitoring:** * Ensure security-relevant events (login failures, unauthorized access attempts) are logged with sufficient detail and alerts are triggered. **Tools:** * **SAST (Static Application Security Testing):** * `Bandit` (Python), `GoSec` (Go), `ESLint` (JS/TS) in CI. * **DAST (Dynamic Application Security Testing):** * `OWASP ZAP`, `Burp Suite` (for penetration testing and vulnerability scanning). * **Network Scanners:** * `Nmap`, `SSL Labs`. * **Dependency Scanners:** * `Snyk`, `Dependabot`, `npm audit`. * **Manual Penetration Testing:** Expert-driven security audits. --- **### 5. Performance Test Cases** * **Objective:** Evaluate the system's responsiveness, stability, scalability, and resource utilization under various load conditions. * **Scope:** Critical API endpoints, blockchain interactions, database operations, and data ingestion from provider agents. **Examples:** * **Load Testing:** * **API Endpoints:** Simulate increasing concurrent users for `POST /auth/login`, `POST /deployments`, `GET /deployments`, `GET /providers`, `POST /deployments/{id}/bids/{id}/accept`. Measure response times, throughput (requests/sec), and error rates. * **Bidding System:** Simulate a large number of providers bidding on a single deployment request, or many deployment requests simultaneously triggering bids. * **Monitoring Data Ingestion:** Simulate thousands of provider agents simultaneously sending metrics and logs to the backend. * **Database Performance:** Test database response under high read/write loads (e.g., concurrent user logins, frequent deployment updates, lease lookups). * **Stress Testing:** * Push the system beyond its normal operating capacity to identify its breaking point and how it recovers. * Observe resource utilization (CPU, memory, network I/O) on backend servers, database, and message queue under extreme load. * **Scalability Testing:** * Verify the system's ability to scale horizontally (e.g., adding more Kubernetes pods for FastAPI services, increasing Kafka partitions) and maintain performance as load increases. * Measure the impact of adding more providers and deployments on overall system performance. * **Latency Measurement:** * Measure end-to-end latency for critical user actions (e.g., tenant deploying an app, accepting a bid, withdrawing AKT). * Measure latency for blockchain transactions (acknowledgement and finality). * **Reliability/Stability Testing:** * Run extended load tests (e.g., 24-48 hours) to detect memory leaks, resource exhaustion, or other long-term stability issues. * **Network Performance:** * Test network latency and bandwidth impact on communication between backend services and provider agents, especially for log/metric streaming. **Tools:** * **Load/Stress Testing:** * `JMeter`, `Locust` (Python-based), `k6`. * **Monitoring:** * `Prometheus`, `Grafana` (to visualize resource usage,

response times, error rates during tests). * **Profiling:** Python's built-in profilers, Go's ``pprof`` to identify performance bottlenecks in specific code paths. --- **6. Automation Testing Strategy**

Objective: Integrate testing throughout the SDLC to provide rapid feedback, improve code quality, and reduce manual effort. **Strategy Overview:**

- Shift-Left Approach:** Implement testing as early as possible in the development cycle.
- Test Pyramid Model:**
 - Unit Tests (Foundation - ~70%):** Fast, isolated, developer-centric. Run on every code change (pre-commit, CI).
 - Integration Tests (Middle - ~20%):** Verify interactions between components. Run on feature branches and pull requests.
 - API/Service Tests (Top-Middle - ~9%):** Black-box testing of API endpoints. Run on merge to main, and before deployments to staging/production.
 - UI/End-to-End Tests (Top - ~1%):** Simulate user journeys in the browser. Run on staging environments for critical paths.
- CI/CD Integration:** All automated tests will be integrated into the GitHub Actions/GitLab CI pipelines.
- Test Data Management:**
 - Unit/Integration:** Use in-memory databases (e.g., ``sqlite`` for Python) or temporary test databases, with fixtures for setup and teardown. Mock external services.
 - API/E2E:** Seed databases with realistic, anonymous data. Implement clean-up routines to ensure test isolation.
 - Blockchain Interaction:** Use local testnet instances or highly controlled development environments for non-mocked integration tests.
- Environment Management:**
 - Development:** Local environments with Docker Compose for quick iteration.
 - Testing/Staging:** Dedicated Kubernetes clusters with representative data for integration, API, security, and performance testing.
 - Production:** Monitor and perform smoke tests post-deployment.
- Reporting & Feedback:**
 - Integrate test results into CI/CD pipeline reports (e.g., JUnit XML, Allure Reports).
 - Dashboarding (Grafana) for performance test results.
 - Automated alerts for critical test failures.

Automation Workflow:

- Developer Workflow:**
 - Write unit tests alongside code.
 - Run unit tests locally before committing.
 - Implement pre-commit hooks to enforce basic code quality and unit test execution.
- Pull Request (PR) Workflow:**
 - Build:** Docker image build for backend/frontend.
 - Static Analysis:** Linting (ESLint, Flake8), type checking (TypeScript, MyPy), security scanning (Bandit, GoSec).
 - Unit Tests:** Run all unit tests for affected services/applications.
 - Integration Tests:** Run selected integration tests for affected modules (e.g., API <-> DB for new features).
 - Code Coverage:** Report code coverage metrics (e.g., ``Codecov``).
- Merge to `main` Workflow:**
 - Execute all tests run in PR workflow.
 - Full Integration Tests:** Run comprehensive integration tests, potentially involving mocked blockchain interactions or a dedicated internal testnet.
 - API Tests:** Execute full API test suite (Postman/Newman or ``pytest-httpx``).
 - Deployment to Staging:** If all tests pass, deploy to a staging environment.
- Staging Environment Workflow:**
 - UI/E2E Tests:** Run ``Cypress``/``Playwright`` tests for critical user journeys (e.g., tenant deployment, provider registration, wallet operations).
 - Security Scans:** DAST scans (``OWASP ZAP``) against the deployed staging environment.
 - Performance Tests:** Run selected load tests against critical APIs.
 - Manual QA:** Exploratory testing, usability testing, and edge-case validation.
- Production Deployment:**
 - Smoke Tests:** A small suite of critical API and UI tests to ensure basic functionality after deployment.
 - Post-Deployment Monitoring:** Continuous monitoring of logs, metrics, and application health.

Chosen Automation Tools:

- Unit Testing:** ``pytest`` (Python), ``go test`` (Go), ``Jest``/``React Testing Library`` (Frontend).
- API Testing:** ``pytest`` with ``httpx``, ``Postman``/``Newman``.
- Integration Testing:** ``pytest``, Docker Compose, ``testcontainers`` (for ephemeral dependencies).
- UI/E2E Testing:** ``Cypress`` or ``Playwright``.
- Performance Testing:** ``Locust`` (Python), ``JMeter``.
- Security Testing:** ``Bandit``, ``GoSec``, ``Snyk``, ``OWASP ZAP``.
- CI/CD:** GitHub Actions.
- Reporting:** Allure Reports, ``Codecov``.

This comprehensive plan aims to establish a robust QA process, ensuring the Akash project delivers a reliable, secure, and performant decentralized cloud marketplace.